Decision Record: Multi-Bike Reservations

Nasar Roble | MSSE 601: Software Engineering Fundamentals | Golden Bike Shop

1. Context

As the Week 6 data-design deliverable owner, I drove the review of how the reservation data should represent real rental activity. Golden Bike Shop serves four stores and a 400-bike fleet, and its requirements call for rental-period conflict prevention across locations (Week 3 FR-04 and NFR-02). A customer may reserve more than one bike, while each bike can have its own rental period and agreed rate. The Week 5 model stored a single bike ID directly on RentalReservation, so it could not represent a multi-bike reservation cleanly. The eight-week course schedule and four-person team also meant the model had to remain practical to implement and review. The decision was how to represent each bike and enforce overlap rules without weakening the shared booking record.

2. Options

Option A - Keep one bike ID on RentalReservation. This is the simplest structure and keeps common reservation details in one record, but a reservation containing several bikes would require multiple independent reservations or duplicated customer and booking information. It also leaves bike-specific periods and rates awkward to represent.

Option B - Create a separate reservation record for every bike. This supports per-bike periods and makes it possible to check each bike independently, but it loses the clear parent record for one customer booking. Shared details could be duplicated and later updates or cancellation handling could become inconsistent.

Option C - Keep RentalReservation as the parent and add ReservationItem as a child record for each bike. Each item stores the bike, its reserved period, and agreed rate. This adds a relationship and more data-handling steps, but represents both the overall booking and each bike-level commitment.

3. Decision

I selected Option C: retain RentalReservation as the booking-level record and introduce ReservationItem for each bike included in the reservation. The overlap constraint belongs at the ReservationItem level, where the bike and requested time interval can be evaluated together.

4. Reasoning

This structure maps directly to the business rule rather than forcing the business to fit a simpler schema. FR-04 requires the system to reject a reservation when the requested period conflicts with an existing reservation; NFR-02 requires that concurrent requests do not create duplicate confirmed bookings. A child row per bike gives the database a precise place to enforce that rule for each bike and interval. It also supports one reservation containing multiple bikes without duplicating the parent booking details. Keeping the agreed rate on the item preserves the price that applied when the bike was reserved, even if the shop changes its rate later. These needs were confirmed in Week 6 by checking the model against prewritten edge cases, rather than judging it by how complete the diagram looked.

5. Consequences

The main benefit is a data model that can represent multi-bike bookings and enforce availability at the correct level. The trade-off is added schema and application complexity: reservation reads and writes must join or create item records, and cancellation, payment, and notification flows must keep the parent and children consistent. The overlap rule also needs concurrency testing; a well-structured schema does not prove that the implementation is correct. I would revisit the design if the business required cross-bike atomicity rules that are not captured at item level, or if query and maintenance costs grew enough to outweigh the clearer representation. Before treating this decision as verified, I would run overlapping and simultaneous reservation cases and inspect persisted results. A design decision alone is not test evidence.

Evidence

Week owned and deliverable: Week 6 - Data Design (deliverable owner: Nasar Roble).

Pointer in deliverable: Section 4.1, “Making a reservation that covers several bikes”; Section 5.6, “What changed in our design”; Section 6.1, “Major Team Decisions” (ReservationItem).

AI use: ChatGPT supplied a simplified rental data model as an independent starting point. I did not accept it as final: I reviewed it against the team’s independently written edge cases and retained the multi-bike, per-bike-period design by adding ReservationItem (and related item-level structures) where the generated model was insufficient.